Vetto AI - Comprehensive System Architecture

1. Executive Summary

Vetto is a cutting-edge, multi-modal AI-powered first-round technical interviewer. It conducts realistic, resume-aware voice interviews using a live LLM agent, evaluates candidate responses, tracks visual engagement, executes code in the browser, and produces structured performance reports.

This project goes far beyond text-based wrappers by combining Realtime Audio, Vision (Face Tracking), Client-Side Code Sandboxing, and highly structured Graph-based LLM workflows.

2. System Architecture Overview

The system is decoupled into three primary components:

- Frontend (Vite + React): Custom CSS Glassmorphism, Pyodide (WASM) for Python execution, sandboxed iframes for JS, MediaPipe for face tracking, and LiveKit for WebRTC audio.
- Backend (FastAPI + Python): REST endpoints for preparation, in-memory state management, LangGraph orchestration, and JSON state caching.
- Realtime Agent Worker (LiveKit): Runs google.realtime.RealtimeModel with Server-side Voice Activity Detection (VAD) for natural human interruptions (barge-in).

3. Deep Dive: LangGraph AI Pipeline

The preparation phase runs an autonomous graph of specialized Gemini agents:

- ResumeParser: Extracts skills, experience, and projects from the PDF via Gemini API.
- JDParser: Parses the Job Description to extract required competencies.
- GapAnalyzer: Compares the Resume vs JD to identify Skill Gaps.
- QuestionPlanner: Generates a custom 12-question interview plan designed specifically to test gaps and validate strengths.

4. Deep Dive: Realtime Voice & Personas

The LiveKit agent is injected with dynamic context instead of reading a static script. It supports dynamic Personas (Strict HR, Friendly HR, Technical) which drastically alters its system instructions.

Localization (Urdu-English Mix): For South Asian candidates, the agent is instructed to dynamically translate the approved English questions into a fluent mix of Urdu grammar and English technical terms on the fly.

5. Multimodal Coaching Analytics

Candidates are evaluated on multiple axes that text-only LLMs cannot see:

- Body Language (Vision): Google MediaPipe FaceLandmarker calculates the nose-to-eye ratio to track exactly how often the candidate breaks eye contact with the screen.
- Speech Analytics (Audio): The backend runs regex on the final transcript to count filler words (um, uh, like) and computes accurate Words Per Minute (WPM) based on the session duration.
- STAR Method Detection (Reasoning): Gemini 1.5 Pro explicitly parses behavioral answers to verify Situation, Task, Action, and Result, flagging exactly which parts the candidate forgot.

6. Code Execution & Integrity

To prevent cheating while executing code securely:

- Sandboxed Execution: Python runs via Pyodide (WebAssembly), meaning infinite loops cannot crash the server. JS runs in invisible iframes overriding console.log.
- Proctoring System: Detects tab-switching via the visibilitychange API. MediaPipe detects if there are 0 faces or multiple faces in the frame. A 3-strike system automatically halts the interview.

7. Reporting & PDF Generation

Once the interview completes, FinalAnalyzer synthesizes Technical accuracy, Code tests, Integrity flags, Speech WPM, and Body Language into a master JSON object.

report_generator.py parses this into a Markdown report, which the React frontend clones and renders into a downloadable styled PDF via html2pdf.js.